

Operating Systems

Youjip Won

KAIST



43. Log-structured File Systems

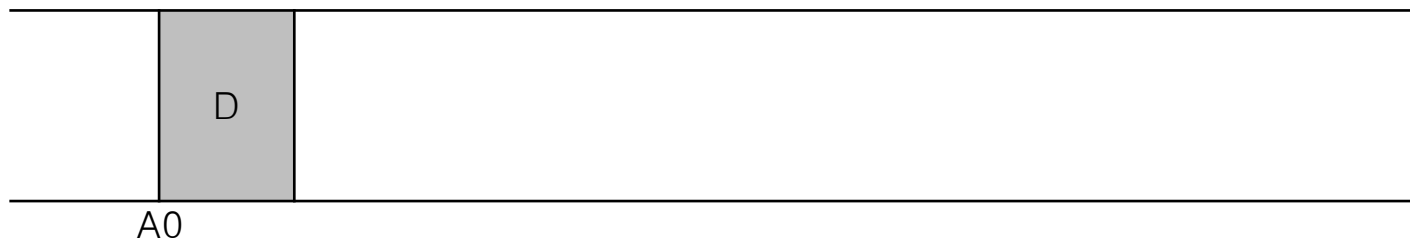
Overview

- In the early 90`s, a new file system known as the log-structured file system(LFS) was developed.
- The Motivation ...
 - ◆ Memory sizes were growing.
 - ◆ Large gap between random IO and sequential IO performance.
 - ◆ Existing File System perform poorly on common workloads.
 - ◆ File System were not RAID-aware. There exists small write problem.
- In this chapter, we study Log-Structured Filesystem(LFS).
 - ◆ How can a file system **transform all writes into sequential writes** ?

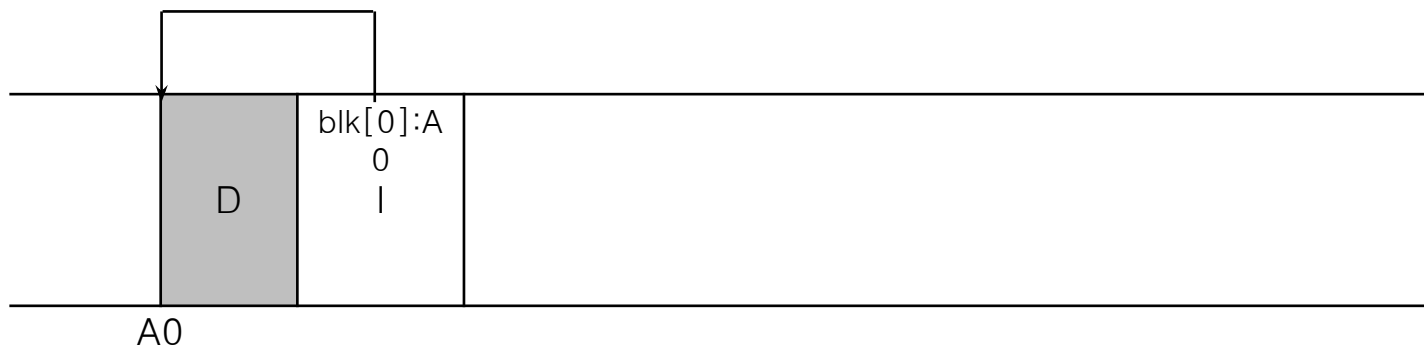
Writing to Disk Sequentially

- How do we transform all updates to file-system state into a series of sequential writes to disk?

- ◆ data update

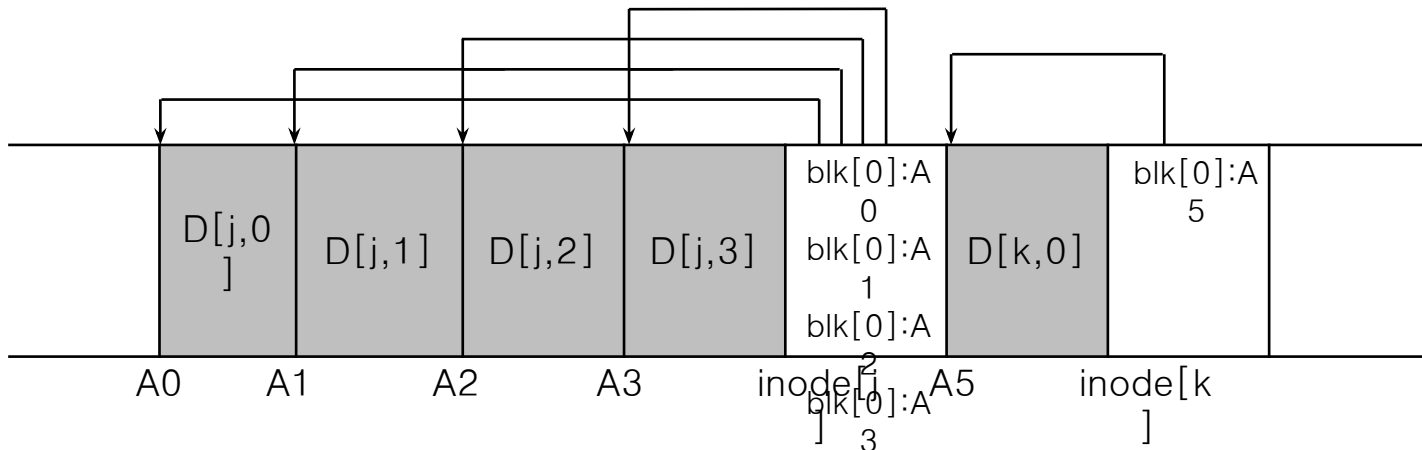


- ◆ metadata needs to be updated too. (Ex. inode)



Segment

- Writing to the disk sequentially is not enough to guarantee the efficient writes.
 - ◆ Disk may rotate between the writes. □ lose a single revolution between the writes.
- Write buffering.
 - ◆ Segment: a set of sequential writes that are written to the disk with a single unit.
 - ◆ Keep track of updates in **memory buffer**. (a few Mbyte)
 - ◆ Write them to disk all at once, when it has sufficient number of updates.



Right segment size

- Time to write D Mbyte

$$T_{write} = T_{position} + \frac{D}{R_{peak}}$$

- Effective write bandwidth

$$R_{effective} = \frac{D}{T_{write}} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} \quad (43.2)$$

- We like to make the effective write bandwidth close to peak bandwidth with some fraction F ($0 < F < 1$)

$$R_{effective} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} = F \times R_{peak}$$

Right segment size

- Then, D can be computed as follows.

$$D = F \times R_{peak} \times (T_{position} + \frac{D}{R_{peak}})$$

$$D = (F \times R_{peak} \times T_{position}) + \left(F \times R_{peak} \times \frac{D}{R_{peak}} \right)$$

$$D = \frac{F}{1-F} \times R_{peak} \times T_{position}$$

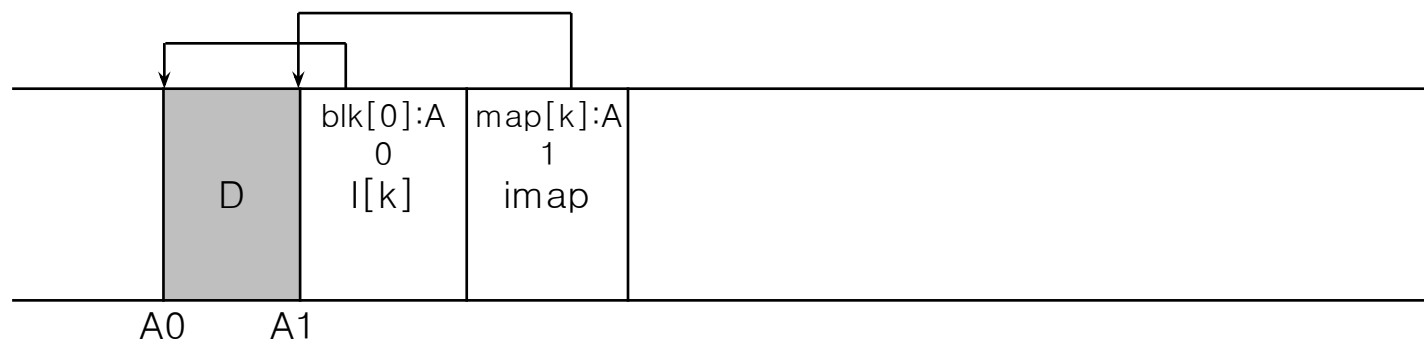
- Example: Positioning time 10 msec, peak transfer rate 100MByte/sec, we like to achieve 90% of the peak rate

$$D = 0.9 \times 0.1 \times 100 \text{ Mbyte/sec} \times 0.01 \text{ secs} = 9 \text{ Mbyte}$$

- What is D if $F = 0.95$?

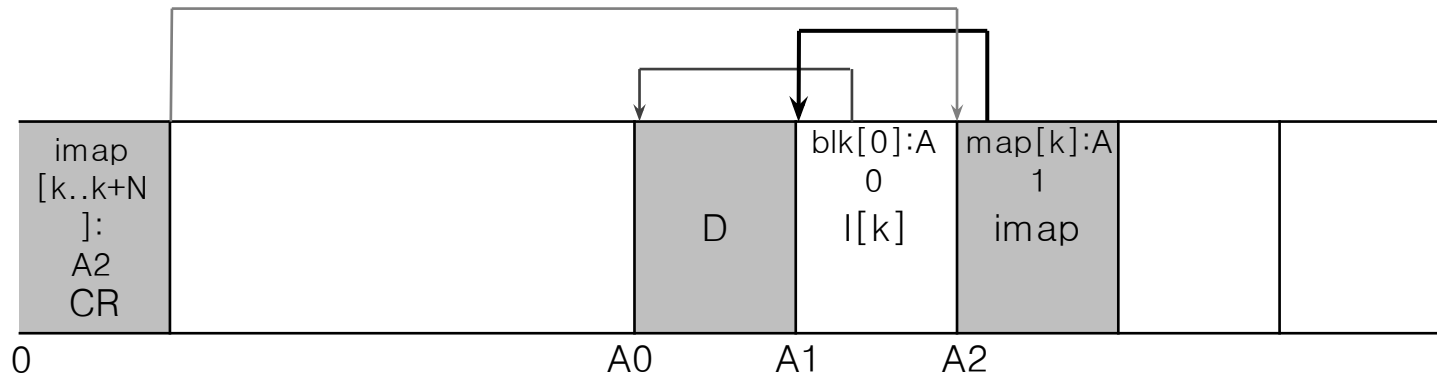
Finding Inode

- The position of the inodes keep changing.
- The Inode Map
 - ◆ A data structure that contains the location of the most recent inode for a given inode number.
 - ◆ Places the chunk of updated inode map next to the updated inode.
 - ◆ Where to find the inode map?



The Checkpoint Region

- How to find the inode map spread across the disk?
 - ◆ The LFS File system must have fixed location on disk to begin a file lookup.
- **Checkpoint Region**
 - ◆ fixed location in the LFS partition.
 - ◆ Contain the pointers to the latest of the inode map.



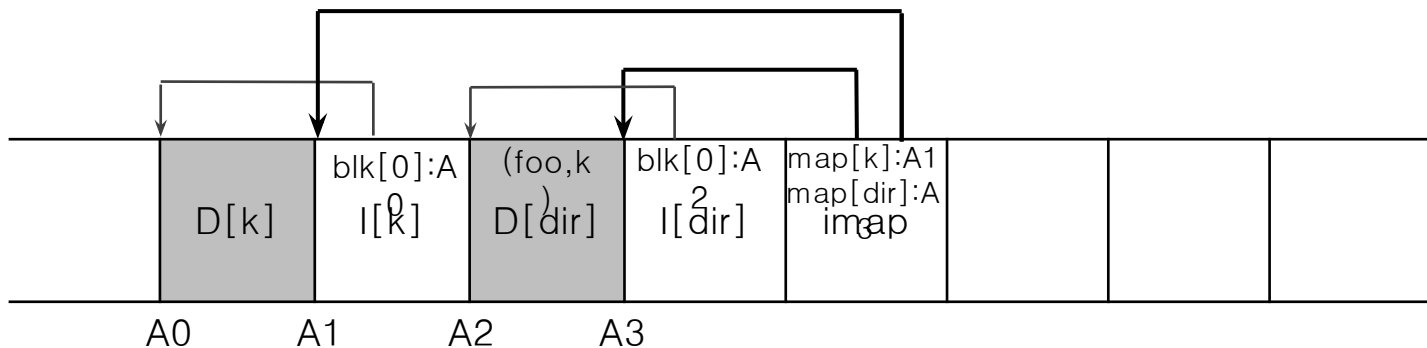
Reading a file from the disk

- Reading a file block
 - ◆ Read a checkpoint region
 - ◆ Read inode map
 - ◆ Read inode
 - ◆ Read data block
- What about sequential read?
 - ◆ It may become random read.

LFS is optimized for the write operation.

What About Directories?

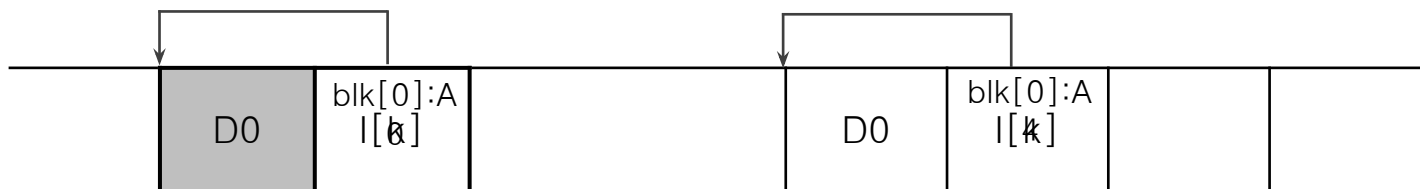
- Directory: a set of <inode, filename>
- How does LFS store directory data?
- Creating a file: `foo`
 - ◆ Update the directory inode. (inode #: `dir`)
 - ◆ Update the directory entry. (`foo`, `k`)
 - ◆ Update inode for the created file. (inode #: `k`)
 - ◆ Update the data block for the created file.



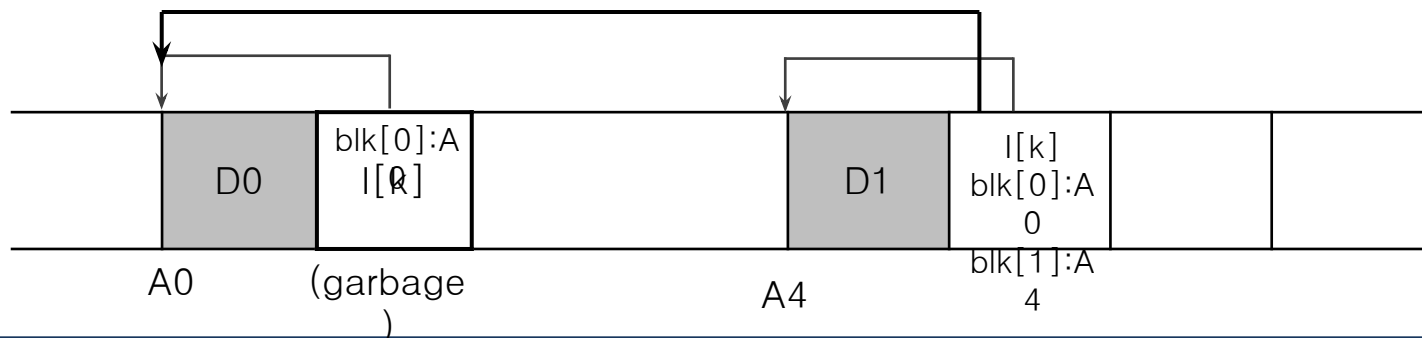
- Recursive update (cascade update issue): the location of the inode keeps changing. □ the associated directory entry can be updated as well. □ solved by inode map.

Garbage

- LFS keeps writing newer version of file.
- Garbage: LFS leaves the older versions of file structures all over the disk.
- An example of garbage
 - ◆ Overwrite the data block:



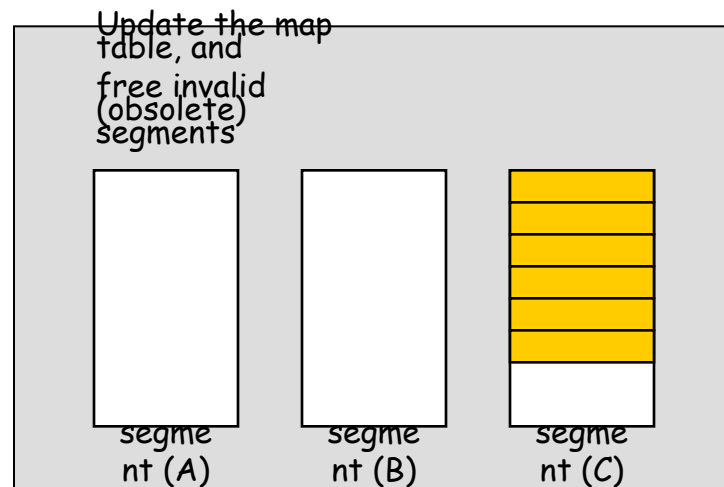
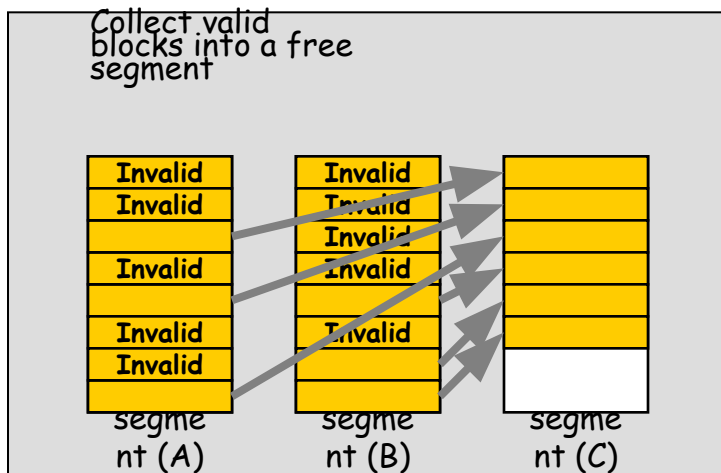
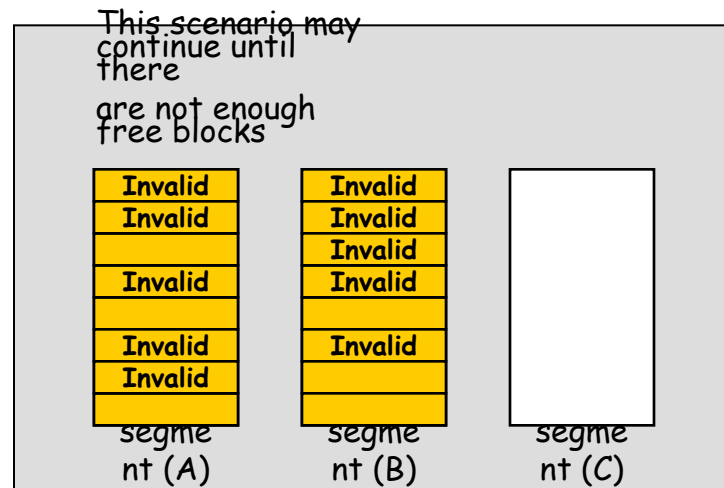
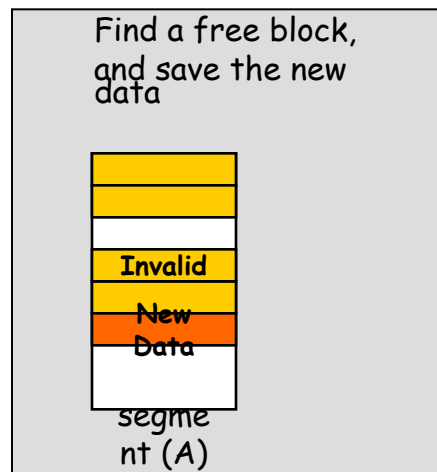
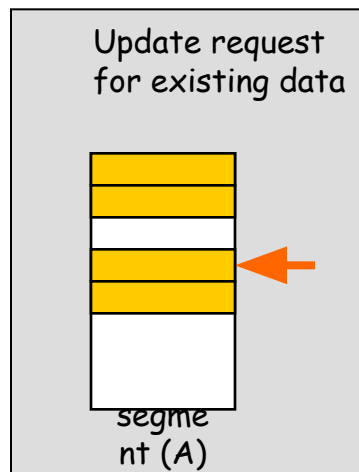
- ◆ Append a block to that original file k:



Garbage Collection (segment cleaning)

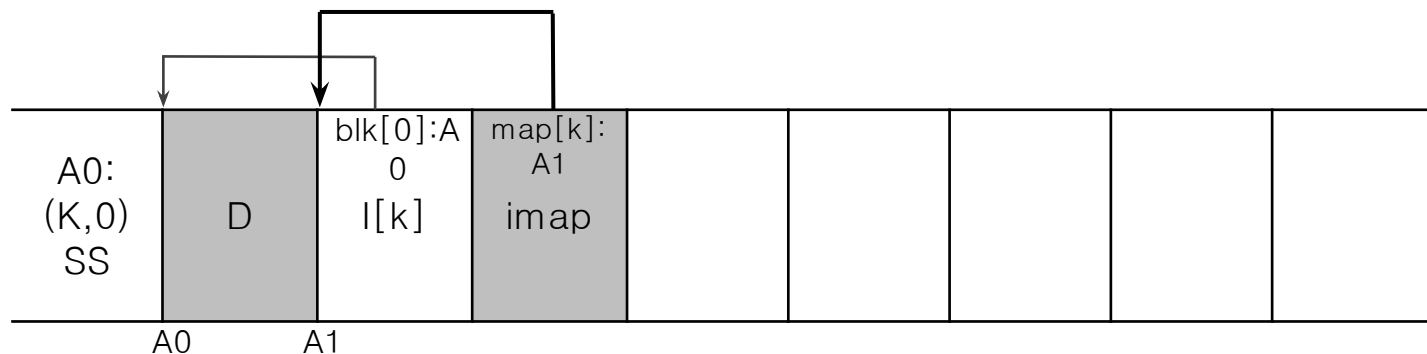
- What to do with the older versions of the block
 - ◆ Versioning filesystem: keep the old blocks and allow the users to restore to the older version of the filesystem status.
 - ◆ LFS: periodically clean the older versions of the file data, inodes and other structures.
- Unit of garbage collection: Segment
 - ◆ Reads a number of old segments, M segments.
 - ◆ Identify the valid blocks.
 - ◆ Write them to a number of new segments (in memory), N segments.
 - ◆ Write N segments to the disk.
 - ◆ Then, $N < M$.

Garbage collection



Segment Summary Block

- Store the inode and the file offset for each data block in it.
- In garbage collection, we need to identify the obsolete blocks.
- Compare the block address of file K offset 0 based upon the Segment Summary and based upon the in-memory imap. If they coincide, the block is alive. □ expensive



Issues in garbage collection

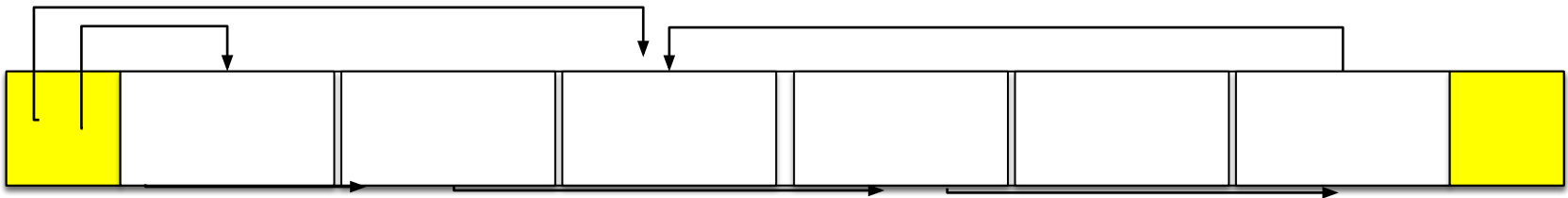
- When to clean
 - ◆ Periodically
 - ◆ When a system is idle
 - ◆ When the disk is full
- Which block to consolidate?
 - ◆ Hot segment: the blocks are updated periodically
 - ◆ Cold segment: the blocks are not updated.
 - ◆ Hot segment: clean later.
 - ◆ Cold segment: clean sooner.

Crash recovery

- What if the crash happens when the LFS is in the middle of writing the segment to the disk?
- LFS maintains a set of segments as a linked list in memory



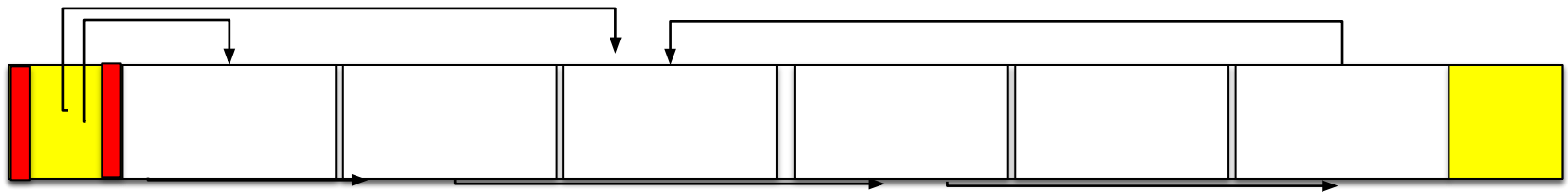
- LFS organizes the filesystem partition as a log (a linked list of the segments).
 - ◆ Two checkpoint regions: one at the beginning and the one at the end.



Crash recovery

□ Consistent Update on CR

- ◆ Write timestamp at the beginning of CR.
- ◆ Write CR body.
- ◆ Write time stamp at the end of the CR.
- ◆ When crash occurs, chooses the most recent CR with valid consistent time stamps.



□ Crash recovery

- ◆ Read the CR and rebuild imap.
- ◆ Perform roll-forward.
 - Start from the first segment in CR.
 - Scan the valid segment following the “next segment” pointer and update the imap.

Summary

- Introduce a new approach to updating the disk.
 - ◆ **Shadow paging** in database system, **Copy-on-Write** in file system.
- Gather all updates into an in-memory segment.
 - ◆ Write them out together sequentially.
- LFS-style is excellent for performance on many different devices.
 - ◆ Hard drives, parity-based RAIDs, even Flash-based SSDs.
- Some modern commercial filesystems adopt a similar copy-on-write approach even though it generates garbage.
 - ◆ NetApp's **WAFL**, Sun's **ZFS** and Linux **btrfs**
 - ◆ In particular, WAFL turns cleaning problem into a feature, by providing old version of the file system via **snapshot**.